

```

import streamlit as st
import requests
import datetime
import pandas as pd

st.set_page_config(page_title="SisCuratela Pro Gestão e Compliance",
page_icon="", layout="wide")

if "logado" not in st.session_state:
    st.session_state.update({
        "logado": False,
        "token": None,
        "nome_usuario": None,
        "perfil_usuario": None,
        "msg_sucesso": None,
        "ocr_valor": "",
        "ocr_desc": "",
        "ocr_ref": "",
        "ocr_estab": "",
        "ocr_data": None # <-- ADICIONE ESTA LINHA
    })

def login_backend(email, senha):
    url_api = "http://127.0.0.1:8000/auth/login"
    try:
        resposta = requests.post(url_api, json={"email": email, "senha": senha},
        timeout=60)
        if resposta.status_code == 200:
            dados = resposta.json()
            return True, dados["nome"], dados["perfil"], dados["access_token"]
        elif resposta.status_code == 401:
            return False, "E-mail ou senha incorretos.", None, None
        else:
            return False, f"Erro no servidor: {resposta.text}", None, None
    except requests.exceptions.Timeout:
        return False, "Erro de Timeout.", None, None
    except requests.exceptions.ConnectionError:
        return False, "Erro Crítico: Backend desligado.", None, None

def enviar_lancamento(dados_transacao, token):
    url_api = "http://127.0.0.1:8000/transacoes/novo"
    headers = {"Authorization": f"Bearer {token}"}
    try:
        resposta = requests.post(url_api, json=dados_transacao, headers=headers)
        if resposta.status_code == 200:
            resultado = resposta.json()
            return True, resultado.get("mensagem", "Sucesso"),
            resultado.get("id_transacao")
        else:
            erro_detalhe = resposta.json().get("detail", "Erro desconhecido.")
            return False, erro_detalhe, None
    except requests.exceptions.ConnectionError:
        return False, "Falha de conexão com o Backend.", None

```

```

# === TELA DE LOGIN SEGURA ===
if not st.session_state["logado"]:
    col1, col2, col3 = st.columns([1, 2, 1])
    with col2:
        st.markdown("<h1 style='text-align: center; color: #2E7D32;'>SisCuratela
Pro</h1>", unsafe_allow_html=True)
        st.markdown("<h3 style='text-align: center; color: gray;'>Sistema de
Gestão e Prestação de Contas (MPDFT)</h3>", unsafe_allow_html=True)
        st.write("---")
        with st.form("form_login"):
            email = st.text_input("E-mail de Acesso")
            senha = st.text_input("Senha de Acesso", type="password")
            submit = st.form_submit_button("Autenticar via Servidor Seguro")
            if submit:
                sucesso, nome_ou_erro, perfil, token =
login_backend(email.strip(), senha)
                if sucesso:
                    st.session_state.update({
                        "logado": True,
                        "nome_usuario": nome_ou_erro,
                        "perfil_usuario": perfil,
                        "token": token
                    })
                    st.rerun()
                else:
                    st.error(f"Falha na autenticação: {nome_ou_erro}")

# === PAINEL LOGADO (ENTERPRISE) ===
else:
    st.sidebar.title("Painel de Controle")
    st.sidebar.markdown(f"**Operador:** {st.session_state['nome_usuario']}")
    st.sidebar.markdown(f"**Perfil:** {st.session_state['perfil_usuario']}")
    st.sidebar.write("---")
    menu = st.sidebar.radio("Navegação", ["Painel Central", "Novo Lançamento
Financeiro", "Extratos e Relatórios"])

    if st.sidebar.button("Encerrar Sessão (Logout)"):
        st.session_state.clear()
        st.rerun()

    if menu == "Painel Central":
        st.title("Painel Executivo e Compliance")
        st.success("Conexão estabelecida com segurança entre Frontend, Backend e
PostgreSQL!")
        st.info("Utilize o menu lateral para registrar transações financeiras
com validação automática para a prestação de contas.")

    elif menu == "Novo Lançamento Financeiro":
        st.title("Novo Lançamento Financeiro e Leitura Inteligente (OCR/IA)")
        st.markdown("Envie um arquivo ou use a câmera do seu celular para
fotografar notas fiscais, recibos e boletos em tempo real.")

    if st.session_state.get("msg_sucesso"):
        st.success(st.session_state["msg_sucesso"])

```

```

st.session_state["msg_sucesso"] = None

if "form_counter" not in st.session_state:
    st.session_state["form_counter"] = 0

# --- SEÇÃO DE CAPTURA INTELIGENTE (UPLOAD OU CÂMERA DO CELULAR) ---
with st.expander(" Assistente de Captura e Leitura IA (Upload ou
Câmera)", expanded=True):
    tipo_entrada = st.radio(
        "Selecione o método de captura do documento:",
        ["Enviar Arquivo (PDF, PNG, JPG)", " Usar Câmera do Celular /
Webcam"],
        horizontal=True
    )

    arquivo_capturado = None
    if tipo_entrada == "Enviar Arquivo (PDF, PNG, JPG)":
        arquivo_capturado = st.file_uploader("Selecione o documento
fiscal", type=["pdf", "png", "jpg", "jpeg"], key="uploader_ia")
    else:
        st.markdown("Posicione o documento em frente à câmera do seu
dispositivo e clique em tirar foto:")
        arquivo_capturado = st.camera_input("Tirar Foto do Comprovante")

    if arquivo_capturado is not None:
        if st.button("🔍 Processar Documento com IA"):
            with st.spinner("Analisando metadados e estruturando dados
financeiros..."):
                headers = {"Authorization": f"Bearer
{st.session_state['token']}"}}
                files = {"file": (getattr(arquivo_capturado, 'name',
'foto_camera.jpg'), arquivo_capturado.getvalue(), getattr(arquivo_capturado,
'type', 'image/jpeg'))}
                resp_ocr =
requests.post("http://127.0.0.1:8000/transacoes/extrair-ia", files=files,
headers=headers)

                if resp_ocr.status_code == 200:
                    res_json = resp_ocr.json()
                    st.session_state["ocr_valor"] =
res_json.get("valor_sugerido", "")
                    st.session_state["ocr_desc"] =
res_json.get("descricao_sugerida", "")
                    st.session_state["ocr_ref"] =
res_json.get("documento_referencia_sugerido", "")
                    st.session_state["ocr_estab"] =
res_json.get("estabelecimento_identificado", "")
                    st.session_state["ocr_data"] =
res_json.get("data_sugerida", None)
                    st.session_state["form_counter"] += 1
                    st.success(f"Sucesso! Estabelecimento:
**{st.session_state['ocr_estab']}** | Valor Sugerido: **R$
{st.session_state['ocr_valor']}**")
                    st.rerun()

```

```

        else:
            st.error(f"Falha na IA! Código:
{resp_ocr.status_code} - Detalhe: {resp_ocr.text}")

        if st.session_state.get("ocr_estab"):
            st.info(f"**Estabelecimento Identificado:**
{st.session_state['ocr_estab']}")

        st.write("---")
        with st.form("form_transacao"):
            col_a, col_b = st.columns(2)
            with col_a:
                contas_oficiais = {
                    "Conta Corrente (Banco do Brasil)": "conta-bb-corrente",
                    "Conta Poupança (Banco do Brasil)": "conta-bb-poupanca",
                    "Conta Benefício (INSS)": "conta-inss-beneficio"
                }
                nome_conta_selecionada = st.selectbox("Conta Bancária de
Origem", list(contas_oficiais.keys()))
                id_conta = contas_oficiais[nome_conta_selecionada]

                val_inicial = st.session_state.get("ocr_valor", "")
                valor_digitado = st.text_input("Valor (R$)", value=val_inicial,
placeholder="0,00", key=f"valor_dinamico_{st.session_state['form_counter']}")

            with col_b:
                data_inicial = datetime.date.today()
                if st.session_state.get("ocr_data"):
                    try:
                        data_inicial =
datetime.datetime.strptime(st.session_state["ocr_data"], "%Y-%m-%d").date()
                    except:
                        pass

                data_transacao = st.date_input(
                    "Data da Transação",
                    value=data_inicial,
                    key=f"data_dinamica_{st.session_state['form_counter']}")

                doc_inicial = st.session_state.get("ocr_ref", "")
                documento_ref = st.text_input("Documento de Referência (Nº Nota
Fiscal / Recibo)", value=doc_inicial,
key=f"doc_dinamico_{st.session_state['form_counter']}")

                categorias_oficiais = {
                    "Acompanhante": "micro-acompanhante", "Água": "micro-agua",
                    "Alimentação": "micro-alimentacao",
                    "Aluguel": "micro-aluguel", "Bancárias": "micro-bancarias",
                    "Brinquedos": "micro-brinquedos",
                    "Calçados": "micro-calcados", "Cama / Banho":
                    "micro-cama-banho", "Cartoriais": "micro-cartoriais",
                    "Casa Geriátrica": "micro-casa-geriatrica", "Cigarros":
                    "micro-cigarros", "Combustíveis": "micro-combustiveis",

```

```

        "Condomínio": "micro-condominio", "Contador": "micro-contador",
"Dentista": "micro-dentista",
        "Despesas Financeiras": "micro-despesas-financeiras",
"Educação": "micro-educacao", "Eletrodomésticos": "micro-eletronicos",
        "Eletrônicos": "micro-eletronicos", "Empréstimo Pago":
"micro-emprestimo-pago", "Energia Elétrica": "micro-energia-eletrica",
        "Enfermagem": "micro-enfermagem", "Estacionamento":
"micro-estacionamento", "Estética": "micro-estetica",
        "Exames": "micro-exames", "Farmácia": "micro-farmacia",
"Fisioterapeuta": "micro-fisioterapeuta",
        "Fonoaudiólogo": "micro-fonoaudiologo", "Fraldas":
"micro-fraldas", "Gás": "micro-gas",
        "Gastos Funerários": "micro-gastos-funerarios", "Higiene
Pessoal": "micro-higiene-pessoal", "Honorários": "micro-honorarios",
        "Hospital": "micro-hospital", "Imóveis": "micro-imoveis",
"Impostos / Taxas": "micro-impostos-taxas",
        "Informática": "micro-informatica", "Internet":
"micro-internet", "Jornais / Revistas": "micro-jornais-revistas",
        "Judiciais": "micro-judiciais", "Lanches": "micro-lanches",
"Lavanderia": "micro-lavanderia",
        "Lazer": "micro-lazer", "Limpeza": "micro-limpeza", "Livros":
"micro-livros",
        "Manutenção": "micro-manutencao", "Material Escolar":
"micro-material-escolar", "Medicamentos": "micro-medicamentos",
        "Médico": "micro-medico", "Mensalidades": "micro-mensalidades",
"Móveis": "micro-moveis",
        "Nutricionista": "micro-nutricionista", "Obrigações Patronais":
"micro-obrigacoes-patronais", "Papeleria": "micro-papelaria",
        "Perfumaria": "micro-perfumaria", "Plano de Saúde":
"micro-plano-saude", "Produtos Hospitalares": "micro-produtos-hospitalares",
        "Psicólogo": "micro-psicologo", "Psiquiatra":
"micro-psiquiatra", "Refeições": "micro-refeicoes",
        "Reformas": "micro-reformas", "Seguros": "micro-seguros",
"Serviços de Terceiros": "micro-servicos-terceiros",
        "Serviços Domésticos": "micro-servicos-domesticos",
"Supermercado": "micro-supermercado", "Táxi": "micro-taxi",
        "Telefonia": "micro-telefonica", "Terapeuta Ocupacional":
"micro-terapeuta-ocupacional", "Transporte": "micro-transporte",
        "TV": "micro-tv", "Utensílios": "micro-utensilios",
"Vale-Transporte": "micro-vale-transporte",
        "Vestuário": "micro-vestuario", "Veterinário / Pet Shop":
"micro-veterinario-pet-shop", "Outros Pagamentos": "micro-outros-pagamentos"
    }

```

```

    nome_categoria_selecionada = st.selectbox("Categoria da Despesa
(Rubrica Oficial MPDFT)", list(categorias_oficiais.keys()))
    id_micro = categorias_oficiais[nome_categoria_selecionada]

    desc_inicial = st.session_state.get("ocr_desc", "")
    descricao = st.text_area("Descrição Detalhada do Gasto (Proibido
termos genéricos como 'diversos')", value=desc_inicial,
key=f"desc_dinamica_{st.session_state['form_counter']}")

    comprovante_file = st.file_uploader("Anexar Comprovante Fiscal

```

```

Definitivo (Opcional)", type=["pdf", "png", "jpg", "jpeg"])

submit_transacao = st.form_submit_button("Validar e Salvar
Lançamento")

if submit_transacao:
    import re
    if not valor_digitado.strip():
        st.error("Rejeitado pelo Compliance: O campo de valor não
pode estar vazio.")
        st.stop()
    if re.search(r'[a-zA-Z]', valor_digitado):
        st.error("Rejeitado pelo Compliance: O campo de valor não
pode conter letras ou termos inválidos.")
        st.stop()

    partes = valor_digitado.split(',')
    if len(partes) > 2:
        st.error("Rejeitado pelo Compliance: Formato de valor
inválido (múltiplas vírgulas).")
        st.stop()
    if len(partes) > 1 and len(partes[1]) > 2:
        st.error("Rejeitado pelo Compliance: O valor não pode ter
mais de duas casas decimais (centavos).")
        st.stop()

    try:
        valor_limpo = valor_digitado.replace(".", "").replace(",",
".")

        valor_float = float(valor_limpo)
        valor_float = round(valor_float, 2)
    except ValueError:
        st.error("Rejeitado pelo Compliance: Formato de valor
inválido.")
        st.stop()

    payload = {
        "id_conta": id_conta,
        "id_micro": id_micro,
        "data_transacao": str(data_transacao),
        "valor": valor_float,
        "descricao": descricao,
        "documento_referencia": documento_ref
    }

    sucesso_transacao, msg, id_gerado = enviar_lancamento(payload,
st.session_state["token"])

    if sucesso_transacao:
        arquivo_para_anexar = comprovante_file if comprovante_file
is not None else arquivo_capturado
        if arquivo_para_anexar is not None:
            nome_arq = getattr(arquivo_para_anexar, 'name',
'foto_camera.jpg')

```

```

        bytes_arq = arquivo_para_anexar.getvalue()
        tipo_arq = getattr(arquivo_para_anexar, 'type',
'image/jpeg')

        files = {"file": (nome_arq, bytes_arq, tipo_arq)}
        headers = {"Authorization": f"Bearer
{st.session_state['token']}"}
        resp_upload =
requests.post(f"http://127.0.0.1:8000/transacoes/{id_gerado}/anexar",
files=files, headers=headers)

        if resp_upload.status_code == 200:
            st.session_state["msg_sucesso"] = "✅ Lançamento
gravado e Comprovante fotográfico/digital anexado com sucesso!"
        else:
            st.session_state["msg_sucesso"] = "⚠️ Lançamento
gravado, mas falha ao enviar comprovante físico."
        else:
            st.session_state["msg_sucesso"] = f"❌ {msg}"

        st.session_state["ocr_valor"] = ""
        st.session_state["ocr_desc"] = ""
        st.session_state["ocr_ref"] = ""
        st.session_state["ocr_estab"] = ""
        st.session_state["ocr_data"] = None
        st.session_state["form_counter"] += 1
        st.rerun()
    else:
        st.error(f"Rejeitado pelo Compliance: {msg}")

elif menu == "Extratos e Relatórios":
    st.title(" Extrato Consolidado e Inteligência Gerencial")
    st.markdown("Visualização oficial dos lançamentos financeiros
armazenados no PostgreSQL para auditoria, gráficos de controle e conferência do
MPDFT.")
    st.write("---")
    try:
        headers = {"Authorization": f"Bearer {st.session_state['token']}"}
        resposta = requests.get("http://127.0.0.1:8000/transacoes/listar",
headers=headers, timeout=60)
        if resposta.status_code == 200:
            lista_transacoes = resposta.json()
            if lista_transacoes:
                df = pd.DataFrame(lista_transacoes)
                total_geral = df['valor'].sum() if 'valor' in df.columns
            else 0.0

            col_m1, col_m2 = st.columns(2)
            col_m1.metric("Total Lançado no Período", f"R$
{total_geral:,.2f}")
            col_m2.metric("Total de Documentos Registrados", len(df))
            st.write("---")
            st.write("### Visão Gráfica Consolidada")
            if 'id_micro' in df.columns and 'valor' in df.columns:
                col_g1, col_g2 = st.columns(2)
                with col_g1:

```

```

        st.markdown("***Gastos por Categoria (Rubrica
Micro)**")
        df_cat =
df.groupby('id_micro')['valor'].sum().reset_index()
        st.bar_chart(df_cat.set_index('id_micro'))
        with col_g2:
            st.markdown("***Evolução Temporal dos Gastos**")
            df_temp =
df.groupby('data_transacao')['valor'].sum().reset_index()
            st.line_chart(df_temp.set_index('data_transacao'))

        st.write("---")
        st.write("### Lançamentos Detalhados")
        colunas_exibicao = ["data_transacao", "id_conta",
"id_micro", "descricao", "valor", "id_usuario", "comprovante_path"]
        df_exibicao = df[[col for col in colunas_exibicao if col in
df.columns]]
        st.dataframe(df_exibicao, use_container_width=True)

        st.write("---")
        col_exp1, col_exp2 = st.columns(2)
        with col_exp1:
            st.write("### Central de Relatórios Personalizados")
            tipo_relatorio = st.selectbox(
                "Selecione o Tipo de Relatório:",
                [
                    "1. Relatório Analítico Completo (Todas as
Despesas)",
                    "2. Relatório Sintético por Rubrica (Agrupado
por Categoria)",
                    "3. Relatório de Despesas de Alto Valor / Alvará
(> R$ 5.000)",
                    "4. Relatório Filtrado por Período Específico"
                ]
            )
            df_export = df.copy()
            nome_arquivo =
f"relatorio_mpdft_{datetime.date.today()}.csv"
            if "2." in tipo_relatorio and 'id_micro' in df.columns
and 'valor' in df.columns:
                df_export = df.groupby('id_micro').agg(
                    Total_Gasto=('valor', 'sum'),
                    Qtd_Lancamentos=('valor', 'count')
                ).reset_index()
                nome_arquivo =
f"relatorio_sintetico_rubricas_{datetime.date.today()}.csv"
            elif "3." in tipo_relatorio:
                if 'exige_alvara' in df.columns:
                    df_export = df[df['exige_alvara'] == True]
                else:
                    df_export = df[df['valor'] > 5000.0]
                nome_arquivo =
f"relatorio_alto_valor_alvara_{datetime.date.today()}.csv"
            elif "4." in tipo_relatorio:

```



```

        st.markdown("Intervalo de Datas:")
        sub_c1, sub_c2 = st.columns(2)
        with sub_c1:
            d_inicio = st.date_input("Início",
value=datetime.date.today().replace(day=1))
        with sub_c2:
            d_fim = st.date_input("Fim",
value=datetime.date.today())
        df['data_dt'] =
pd.to_datetime(df['data_transacao']).dt.date
        df_export = df[(df['data_dt'] >= d_inicio) &
(df['data_dt'] <= d_fim)]
        nome_arquivo =
f"relatorio_perodo_{d_inicio}_a_{d_fim}.csv"

        csv_data = df_export.to_csv(index=False, sep=';',
decimal=',').encode('utf-8')
        st.download_button(
            label=" Baixar Relatório Selecionado (Excel/CSV)",
            data=csv_data,
            file_name=nome_arquivo,
            mime="text/csv"
        )
    with col_exp2:
        st.write("### Resgate Inteligente de Comprovantes")
        df_com_comprovante =
df[df['comprovante_path'].notnull()]
        if not df_com_comprovante.empty:
            termo_busca = st.text_input(" Buscar por descrição,
data ou categoria:", placeholder="Ex: Farmácia, 186,22...")
            if termo_busca:
                df_filtrado = df_com_comprovante[

df_com_comprovante['descricao'].str.contains(termo_busca, case=False, na=False)
|

df_com_comprovante['data_transacao'].astype(str).str.contains(termo_busca,
na=False) |

df_com_comprovante['id_micro'].str.contains(termo_busca, case=False, na=False)
]
                else:
                    df_filtrado = df_com_comprovante

                    if not df_filtrado.empty:
                        opcoes_comprovantes = {
                            f"{row['data_transacao']} | R$
{row['valor']} | [{row['id_micro']}]": row['id_transacao']
                            for _, row in df_filtrado.iterrows()
                        }
                        transacao_selecionada = st.selectbox("Selecione
o comprovante filtrado:", list(opcoes_comprovantes.keys()))
                        id_selec =
opcoes_comprovantes[transacao_selecionada]

```

```

resp_file =
requests.get(f"http://127.0.0.1:8000/transacoes/{id_selec}/comprovante",
headers=headers)

if resp_file.status_code == 200:
    caminho_orig =
df_com_comprovante.loc[df_com_comprovante['id_transacao'] == id_selec,
'comprovante_path'].values[0]
    extensao = caminho_orig.path if
hasattr(caminho_orig, 'path') else caminho_orig.split('.')[-1]
    st.download_button(
        label=" Baixar Arquivo PDF/Imagem
Selecionado",
        data=resp_file.content,

file_name=f"comprovante_{id_selec}.{extensao}",
mime=resp_file.headers.get('Content-Type')
    )
    else:
        st.error("Arquivo físico não localizado no
servidor.")
    else:
        st.warning("Nenhum comprovante encontrado com o
termo pesquisado.")
    else:
        st.info("Nenhum comprovante anexado aos lançamentos
atuais.")
    else:
        st.info("Nenhum lançamento financeiro registrado no banco de
dados até o momento.")
    else:
        st.error(f"Erro ao carregar os extratos do servidor:
{resposta.text}")
except requests.exceptions.ConnectionError:
    st.error("Erro Crítico: O Backend (FastAPI) parece estar
desligado.")

```